

Trabajo Práctico 3

- Tobias Di Salvo
- Angie Oldano
- Ariel Paz

¿Qué hace el Ejercicio 1?

Se busca predecir el rendimiento académico de un estudiante a partir de tres variables:

Las horas de estudio semanales, el porcentaje de asistencia a clase y las calificaciones obtenidas previamente

Para ello utilizamos dos modelos: **regresión lineal múltiple** y **regresión logística**

Regresión lineal simple y múltiple

Regresión

El análisis por regresión es un método estadístico que consiste en analizar la relación entre dos o más variables de manera tal que el valor de una de las variables pueda ser predicho o explicado usando información proveniente del resto de las variables.

Regresión lineal simple: Hay una sola variable predictora

Regresión lineal múltiple: Hay dos o más variables predictoras.

Regresión lineal simple:

$$y = \beta_0 + \beta_1 x + \varepsilon$$

Regresión lineal múltiple:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k + \varepsilon$$

Conjunto de entrenamiento y de prueba

```
#### 1. Dividir aleatoriamente el conjunto de datos para utilizar una parte de los mismos como conjunto de entrenamiento (el 80 %) ####  
#### y otro como conjunto de prueba (el 20 %). ####  
# Cargar archivo csv  
try:  
    df = pd.read_csv('Estudiantes.csv', encoding='utf-8')  
except UnicodeDecodeError:  
    df = pd.read_csv('Estudiantes.csv', encoding='latin-1')  
  
# Identificar el nombre exacto de la columna de Calificación  
target_col = 'Calificación' if 'Calificación' in df.columns else 'Calificacion'  
  
# Seleccionar las variables independientes (X) y la variable dependiente (y)  
X_data = np.array(df[['Horas de estudio por semana', 'Porcentaje de asistencia', 'Calificaciones previas']])  
X = np.hstack([np.ones((X_data.shape[0], 1)), X_data])  
  
tamano_entrenamiento = int(len(X) * 0.8)  
  
X_entrenamiento_calif = X[:tamano_entrenamiento]  
X_prueba_calif = X[tamano_entrenamiento:]  
  
X_entrenamiento_cond = X_data[:tamano_entrenamiento]  
X_prueba_cond = X_data[tamano_entrenamiento:]  
  
y_prueba_cond = np.array(df['Condición'])[tamano_entrenamiento:]  
y_entrenamiento_cond = np.array(df['Condición'])[:tamano_entrenamiento]  
  
y = np.array(df[target_col])  
y_entrenamiento = y[:tamano_entrenamiento]  
y_prueba = y[tamano_entrenamiento:]
```

Regresión lineal múltiple

```
#### 2. Hallar la ecuación del modelo de regresión lineal múltiple y calcular el coeficiente de determinación R^2 ####
def calcularCoeficientesBeta(X, y):
    # Calcula los coeficientes beta utilizando la fórmula de mínimos cuadrados.
    # Fórmula:  $\beta = (X^T * X)^{-1} * X^T * y$ 
    return np.linalg.inv(X.T @ X) @ X.T @ y

# Calcular los coeficientes beta
beta = calcularCoeficientesBeta(X_entrenamiento_calif, y_entrenamiento)
print(f"### REGRESIÓN LINEAL ###")
print()
print(f"Coeficientes beta: {beta}")
print(f"x1 = Horas de estudio por semana")
print(f"x2 = Porcentaje de asistencia")
print(f"x3 = Calificaciones previas")
print(f"Ecuación del modelo:  $y = \{beta[0]:.6f\} + \{beta[1]:.6f\} * x1 + \{beta[2]:.6f\} * x2 + \{beta[3]:.6f\} * x3$ ")
print()
```

Regresión lineal múltiple

```
def predecirRendimiento(entrada):
    # Predecir el rendimiento del estudiante utilizando los coeficientes beta calculados.
    # Se construye un vector de características con un 1 para el término independiente.
    return entrada @ beta

def sst(y):
    # Calcula la suma de cuadrados totales que representa la variabilidad total de y respecto de su media.
    return np.sum((y - np.mean(y)) ** 2)

def ssr(y, y_pred):
    # Calcula la suma de cuadrados del modelo de regresión que es la variabilidad explicada por el modelo de regresión
    return np.sum((y_pred - np.mean(y)) ** 2)

def sse(y, y_pred):
    # Calcula la suma de cuadrados del error que refleja la variación ocasionada por los errores aleatorios involucrados en el modelo.
    return np.sum((y - y_pred) ** 2)

def calcularR2(y, y_pred):
    # Calcula el coeficiente de determinación R^2, que indica la proporción de la variabilidad total explicada por el modelo.
    return (ssr(y, y_pred) / sst(y))

y_pred_entrenamiento = np.array([predecirRendimiento(entrada) for entrada in X_entrenamiento_calif])
y_pred_prueba = np.array([predecirRendimiento(entrada) for entrada in X_prueba_calif])

r2_entrenamiento = calcularR2(y_entrenamiento, y_pred_entrenamiento)
r2_prueba = calcularR2(y_prueba, y_pred_prueba)
```

Regresión lineal múltiple

Resultados: conjunto de entrenamiento

```
### REGRESIÓN LINEAL ###  
  
Coeficientes beta: [16.07570219  0.27712629  0.22354251  0.23057137]  
x1 = Horas de estudio por semana  
x2 = Porcentaje de asistencia  
x3 = Calificaciones previas  
Ecuación del modelo:  $y = 16.075702 + 0.277126x_1 + 0.223543x_2 + 0.230571x_3$   
  
Conjunto de entrenamiento:  
Predicción para el estudiante 1: 59.76, Valor real: 63.00  
Predicción para el estudiante 2: 54.83, Valor real: 50.00  
Predicción para el estudiante 3: 58.52, Valor real: 55.00  
Predicción para el estudiante 4: 66.97, Valor real: 65.00  
Predicción para el estudiante 5: 62.91, Valor real: 70.00  
Predicción para el estudiante 6: 60.96, Valor real: 61.00  
Predicción para el estudiante 7: 64.61, Valor real: 61.00  
Predicción para el estudiante 8: 53.98, Valor real: 50.00  
Predicción para el estudiante 9: 65.25, Valor real: 65.00  
Predicción para el estudiante 10: 58.03, Valor real: 55.00  
  
SSE: 10017.231023618257  
SSR: 15630.246008183542  
SST: 25647.47703180212  
Coeficiente de determinación  $R^2$  (SSR/SST): 0.6094262600879804  
Coeficiente de determinación  $R^2$  (1 - SSE/SST): 0.6094262600879929
```


Regresión lineal múltiple

Resultados: conjunto de prueba

Conjunto de prueba:

Predicción para el estudiante 1:	58.32,	Valor real:	57.00
Predicción para el estudiante 2:	61.89,	Valor real:	61.00
Predicción para el estudiante 3:	59.90,	Valor real:	64.00
Predicción para el estudiante 4:	67.79,	Valor real:	64.00
Predicción para el estudiante 5:	64.84,	Valor real:	66.00
Predicción para el estudiante 6:	63.18,	Valor real:	69.00
Predicción para el estudiante 7:	53.00,	Valor real:	50.00
Predicción para el estudiante 8:	56.86,	Valor real:	50.00
Predicción para el estudiante 9:	61.67,	Valor real:	63.00
Predicción para el estudiante 10:	56.47,	Valor real:	50.00

SSE: 2804.8517104083166

SSR: 2921.601085367626

SST: 6121.1830985915485

Coeficiente de determinación R^2 (SSR/SST): 0.47729352942242015

Coeficiente de determinación R^2 (1 - SSE/SST): 0.5417794787001718

Predicción de la calificación

Conjunto de entrenamiento:

R^2 (SSR/SST):

0.6094

R^2 (1 - (SSE/SST)):

0.6094

Conjunto de prueba:

R^2 (SSR/SST):

0.4772

R^2 (1 - (SSE/SST)):

0.5417

Regresión Logística

- Encuentra relaciones entre factores para predecir el valor de uno en función del otro.
- A diferencia de la regresión lineal, la respuesta predicha es una probabilidad.
- Permite estimar la probabilidad de que ocurra un evento (1) o no (0), optimizando los coeficientes de regresión.
- Ayuda a comprender relaciones entre datos y prever resultados para una mejor toma de decisiones.

Regresión Logística

¿Por qué no alcanza la regresión lineal?

La variable respuesta es binaria: $y \in \{0, 1\}$ (Reprobado / Aprobado)

Si se proyecta $y \in \{0,1\}$ sobre el logaritmo de los odds, los errores tienden a infinito:

$$\begin{aligned}\ln(0 / (1-0)) &\rightarrow -\infty \\ \ln(1 / (1-1)) &\rightarrow +\infty\end{aligned}$$

Con errores infinitos, minimizar la suma de cuadrados (mínimos cuadrados) no tiene sentido, por lo tanto usamos la estimación por máxima verosimilitud.

Regresión Logística y matriz de confusión

```
#### Ejercicio 1 - Punto 3 ####
modelo_logistico = LogisticRegression()
modelo_logistico.fit(X_entrenamiento_cond, y_entrenamiento_cond)

beta0_log = modelo_logistico.intercept_[0]
beta_log = modelo_logistico.coef_[0]
print(f"Coeeficientes: beta0 = {beta0_log:.6f}, beta1 = {beta_log[0]:.6f}, "
      | f"beta2 = {beta_log[1]:.6f}, beta3 = {beta_log[2]:.6f}")
print(f"p(x) = exp(z) / (1 + exp(z)), con z = {beta0_log:.6f} "
      | f"+ {beta_log[0]:.6f}*x1 + {beta_log[1]:.6f}*x2 + {beta_log[2]:.6f}*x3")
print()

y_pred_log = modelo_logistico.predict(X_prueba_cond)

tp_log, fp_log, tn_log, fn_log = calcularPositivosYnegativos(y_pred_log, y_prueba_cond)
matriz_confusion_log = matrizDeConfusion(y_pred_log, y_prueba_cond)
accuracy_log = accuracy(tp_log, fp_log, tn_log, fn_log)
f1_log = f1_score(tp_log, fp_log, fn_log)

print("Matriz de confusión (regresión logística):")
print(matriz_confusion_log)
print(f"Accuracy: {accuracy_log:.4f}")
print(f"F1-score: {f1_log:.4f}")
print()
```

$$z = 21.331876 - 0.125748(x_1) - 0.111353(x_2) - 0.119360(x_3)$$

Matriz de confusión y métricas

Matriz

Matriz de confusión

Relaciona los casos positivos y negativos predichos con los reales

		Predicciones	
		Positivos	Negativos
Reales	Positivos	Verdaderos Positivos (TP)	Falsos Negativos (FN)
	Negativos	Falsos Positivos (FP)	Verdaderos Negativos (TN)

Métricas de evaluación

Accuracy $(TP+TN) / total$

Recall TP / P

Specificity TN / N

Precision $TP / (TP + FP)$

F1 Score $2 * precision * recall / (precision + recall)$

True Positive Rate (TRP) TP / P

False Positive Rate (FPR) FP / N

Resultados

Matriz

Matriz de confusión

Para el ejemplo realizado

		Predicciones	
		Positivos	Negativos
Reales	Positivos	55	16
	Negativos	19	52

Métricas de evaluación

Accuracy 0.7535

Recall 0.7746...

Specificity 0.7324

Precision 0.7432...

F1 Score 0.7586

True Positive Rate (TRP) 0.7746...

False Positive Rate (FPR) 0.2676

Predicción sobre estudiante

Finalmente, usamos ambos modelos para predecir el caso de un estudiante hipotético con 25 horas de estudio, 58% de asistencia y 68 de nota previa.

La regresión lineal estima una calificación y la regresión logística predice probabilidad de aprobar.

Predicción de la calificación

```
#### Ejercicio 1 - Punto 4 ####
#### Predicción para un estudiante con 25 hs de estudio, 58% asistencia y 68 de calificación previa ####
print("### PREDICCIÓN PARA NUEVO ESTUDIANTE (Ejercicio 1, punto 4) ###")
print()
nuevo_estudiante = np.array([25, 58, 68]) # x1=horas, x2=asistencia, x3=calif. previa

# Con el modelo de regresión lineal (se antepone el 1 para el término independiente)
nuevo_estudiante_lineal = np.hstack([1, nuevo_estudiante])
prediccion_calificacion = predecirRendimiento(nuevo_estudiante_lineal)
print(f"Regresión lineal → Calificación estimada: {prediccion_calificacion:.2f}")

# Con el modelo de regresión logística. Se busca el índice de 'Aprobado' dentro de
# modelo_logistico.classes_ para leer la probabilidad correcta (sklearn ordena las
# clases alfabéticamente, no necesariamente en el orden que uno espera).
indice_aprobado = list(modelo_logistico.classes_).index('Aprobado')
prob_aprobado = modelo_logistico.predict_proba(nuevo_estudiante.reshape(1, -1))[0][indice_aprobado]
condicion = modelo_logistico.predict(nuevo_estudiante.reshape(1, -1))[0]
print(f"Regresión logística → Probabilidad de aprobar: {prob_aprobado:.4f} "
      f"||| f" → Condición: {condicion}")
print()
```

Predicción de la calificación

Regresión lineal:

Calificación estimada:

51.65

Regresión logística:

Probabilidad de aprobar:

0.0263 -> 2.63%

La regresión lineal estima una calificación de 51.65, y la regresión logística predice que probablemente repruebe (probabilidad de aprobar de solo 2.6%).

¿Qué hace el Ejercicio 2?

SVM

SVM busca la recta o hiperplano que mejor separa las dos clases maximizando el margen entre ambos grupos.

Hiperplano

$$h(\mathbf{x}) := \langle \mathbf{w}, \mathbf{x} \rangle + \beta = 0,$$

Producto interno

$$\langle \mathbf{a}, \mathbf{b} \rangle = a_1 b_1 + a_2 b_2 + \cdots + a_p b_p = \sum_{j=1}^p a_j b_j \in \mathbb{R}.$$

SVM: Kernels y parámetro C

Parámetro C

Controla la penalización por clasificar mal una observación durante el entrenamiento (margen suave)

C grande → menos tolerancia a errores, margen más angosto

C chico → más tolerancia a errores, margen más ancho

C:

0.1

C:

1

C:

0.1

SVM: Kernels y parámetro C

Kernel

El kernel define qué forma puede tener esa frontera de separación. Se probaron 4 tipos:

Lineal:

$$K(a, b) = \langle a, b \rangle$$

Radial:

$$K(a, b) = \exp(-\gamma \|a - b\|^2)$$

Polinómico:

$$K(a, b) = (\langle a, b \rangle + r)^d$$

Sigmoideo:

$$K(a, b) = \tanh(\gamma \langle a, b \rangle + r)$$

Función Kernel

$$K(a, b) = \phi(a)^t \phi(b).$$

→ donde ϕ es la transformación a ese espacio de más dimensiones

Implementación

```
#### Ejercicio 2 ####
#### 1. Aplicar el método de SVM para clasificar los ejemplos del conjunto de prueba, utilizando
#### diferentes kernels y diferentes valores del parámetro C, costo de violación de restricción.
#### (Este punto puede realizarse con bibliotecas específicas.)

kernels = ['linear', 'poly', 'rbf', 'sigmoid']
valores_C = [0.1, 1.0, 10.0]

resultados_svm = []

for kernel in kernels:
    for C in valores_C:
        modelo_svm = svm.SVC(kernel=kernel, C=C)
        modelo_svm.fit(X_entrenamiento_cond, y_entrenamiento_cond)
        y_pred_svm = modelo_svm.predict(X_prueba_cond)

        #### Ejercicio 2 - Punto 2 ####
        #### Matriz de confusión, accuracy y F1-score para cada modelo ####
        tp_svm, fp_svm, tn_svm, fn_svm = calcularPositivosYnegativos(y_pred_svm, y_prueba_cond)
        matriz_confusion_svm = matrizDeConfusion(y_pred_svm, y_prueba_cond)
        accuracy_svm = accuracy(tp_svm, fp_svm, tn_svm, fn_svm)
        f1_svm = f1_score(tp_svm, fp_svm, fn_svm)

        resultados_svm.append({
            'kernel': kernel,
            'C': C,
            'accuracy': accuracy_svm,
            'f1_score': f1_svm,
            'matriz_confusion': matriz_confusion_svm
        })

print(f"--- Kernel: {kernel} | C: {C} ---")
for i in range(min(10, len(X_prueba_calif))):
    print(f" Predicción SVM para el estudiante {i + 1}: {y_pred_svm[i]}, "
          f"Valor real: {y_prueba_cond[i]}")
print(f" Matriz de confusión:\n{matriz_confusion_svm}")
print(f" Accuracy: {accuracy_svm:.4f}")
print(f" F1-score: {f1_svm:.4f}")
print()
```

Ejemplo de resultado

Lineal, C=0.1

```
--- Kernel: linear | C: 0.1 ---  
Predicción SVM para el estudiante 1: Reprobado, Valor real: Reprobado  
Predicción SVM para el estudiante 2: Aprobado, Valor real: Aprobado  
Predicción SVM para el estudiante 3: Aprobado, Valor real: Aprobado  
Predicción SVM para el estudiante 4: Aprobado, Valor real: Aprobado  
Predicción SVM para el estudiante 5: Aprobado, Valor real: Aprobado  
Predicción SVM para el estudiante 6: Aprobado, Valor real: Aprobado  
Predicción SVM para el estudiante 7: Reprobado, Valor real: Reprobado  
Predicción SVM para el estudiante 8: Reprobado, Valor real: Reprobado  
Predicción SVM para el estudiante 9: Aprobado, Valor real: Aprobado  
Predicción SVM para el estudiante 10: Reprobado, Valor real: Reprobado  
Matriz de confusión:  
[[55, 16], [20, 51]]  
Accuracy: 0.7465  
F1-score: 0.7534
```

Ejemplo de resultado

Radial, C=10.0

```
--- Kernel: rbf | C: 10.0 ---  
Predicción SVM para el estudiante 1: Reprobado, Valor real: Reprobado  
Predicción SVM para el estudiante 2: Aprobado, Valor real: Aprobado  
Predicción SVM para el estudiante 3: Aprobado, Valor real: Aprobado  
Predicción SVM para el estudiante 4: Aprobado, Valor real: Aprobado  
Predicción SVM para el estudiante 5: Aprobado, Valor real: Aprobado  
Predicción SVM para el estudiante 6: Aprobado, Valor real: Aprobado  
Predicción SVM para el estudiante 7: Reprobado, Valor real: Reprobado  
Predicción SVM para el estudiante 8: Reprobado, Valor real: Reprobado  
Predicción SVM para el estudiante 9: Aprobado, Valor real: Aprobado  
Predicción SVM para el estudiante 10: Reprobado, Valor real: Reprobado  
Matriz de confusión:  
[[62, 9], [23, 48]]  
Accuracy: 0.7746  
F1-score: 0.7949
```


¿Qué hace el Ejercicio 3?

Objetivo

Comparamos todos los modelos de clasificación que construimos, la regresión logística del Ejercicio 1 y las 12 variantes de SVM del Ejercicio 2

```
### EJERCICIO 3: COMPARACIÓN DE MODELOS DE CLASIFICACIÓN ###  
  
Modelo          Accuracy    F1-score  
Regresión logística  0.7535    0.7586  
SVM (linear, C=0.1)  0.7465    0.7534  
SVM (linear, C=1.0)  0.7535    0.7619  
SVM (linear, C=10.0) 0.7535    0.7619  
SVM (poly, C=0.1)    0.7535    0.7552  
SVM (poly, C=1.0)    0.7606    0.7639  
SVM (poly, C=10.0)   0.7606    0.7671  
SVM (rbf, C=0.1)     0.7676    0.7755  
SVM (rbf, C=1.0)     0.7465    0.7600  
SVM (rbf, C=10.0)    0.7746    0.7949  
SVM (sigmoid, C=0.1) 0.3169    0.0202  
SVM (sigmoid, C=1.0) 0.3169    0.0202  
SVM (sigmoid, C=10.0) 0.3169    0.0202
```

Resultados

Regresión Logística

Accuracy

0.7535

F1 Score

0.7586

SVM

Linear, C=0.1

Accuracy

0.7465

F1 Score

0.7534

Poly, C=0.1

Accuracy

0.7535

F1 Score

0.7552

Radial, C=0.1

Accuracy

0.7676

F1 Score

0.7755

Sigmoideo, C=0.1

Accuracy

0.3169

F1 Score

0.0202

Linear, C=1.0

Accuracy

0.7535

F1 Score

0.7619

Poly, C=1.0

Accuracy

0.7606

F1 Score

0.7639

Radial, C=1.0

Accuracy

0.7465

F1 Score

0.7600

Sigmoideo, C=1.0

Accuracy

0.3169

F1 Score

0.0202

Linear, C=10.0

Accuracy

0.7535

F1 Score

0.7619

Poly, C=10.0

Accuracy

0.7606

F1 Score

0.7671

Radial, C=10.0

Accuracy

0.7746

F1 Score

0.7949

Sigmoideo, C=10.0

Accuracy

0.3169

F1 Score

0.0202

Conclusión

Radial, C=10.0

Accuracy

0.7746

F1 Score

0.7949

El modelo recomendado es el SVM con kernel Radial y C=10.0, con el mayor F1-score y el mayor accuracy.

¿Preguntas?

Gracias por escucharnos.

